Neural Networks

MTC Neural Network Workshop · Spring '26"

TABLE OF CONTENTS

01 What is a Neural Network?

02 What you need to build

03 Activation Functions

04 Building the Network

05 Training & Backpropogation

06 Experiment & Explore

What is a Neural Network

Artificial Intelligence: Any machine/algorithm doing something that normally requires human thinking/input. Ex: playing chess, translating languages, image / facial recognition

Machine Learning: A more specific approach inside AI. Instead of writing specific rules(like an if else statement), the machine learns the rules by itself from examples. (ex: feeding a model a large set of data, until it figures out a pattern)

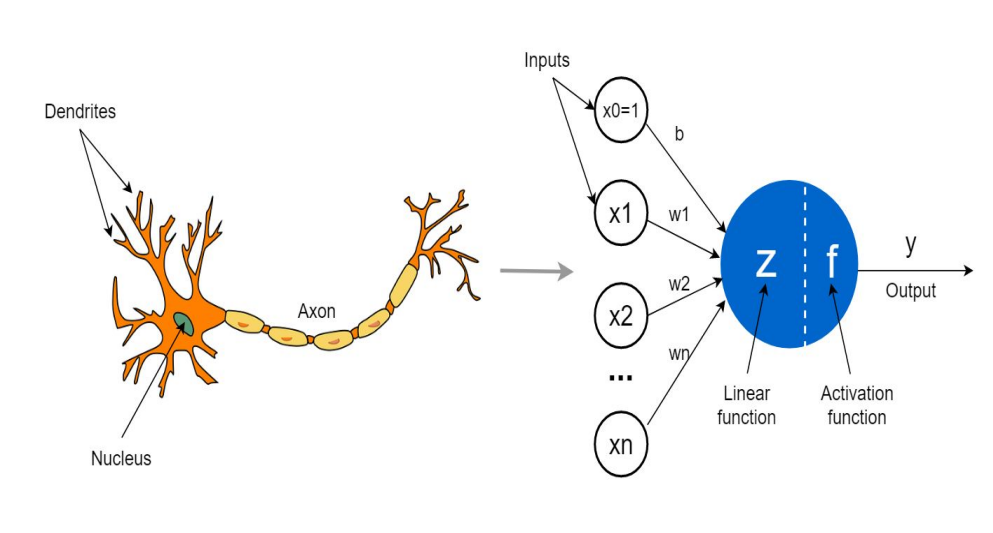
Neural Network: A specific machine learning model. Kind of inspired by how neurons in the brain connect and fire -> lots of small units, connected together, can collectively learn really complex patterns

Powers most LLM's today + image recognition on FaceID, social media feeds, to even medical diagonosis tools

The Neuron

A basic computational unit takes in some numbers, does a small math operation, and then outputs a value

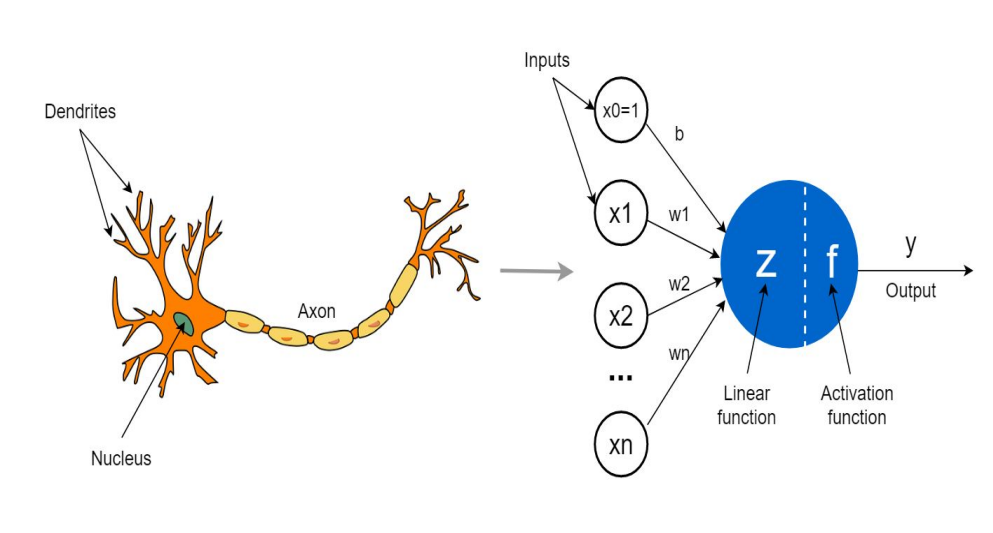
- **Inputs**: raw data, could be from a dataset that may include measurements such as tumor radius, texture, smoothness, etc...



The Neuron

Weights: how much attention the neuron pays to each input. Higher the weight → input is more significant, lower weights is mostly ignored (the NN usually learns this over time)

Bias: an single extra number that gets added after the weighted sum, allowing for the neuron to shift the output up or down (more flexibility)

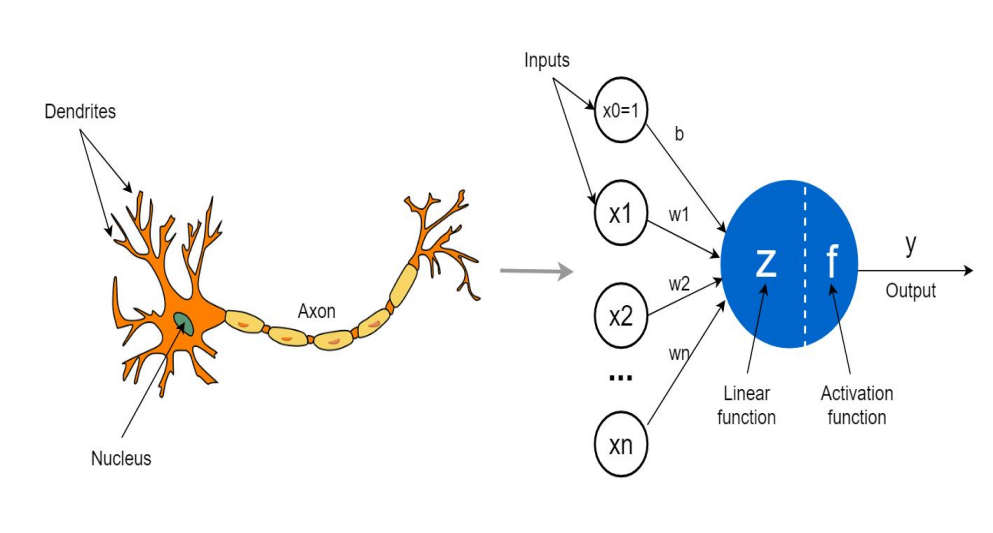


The Neuron

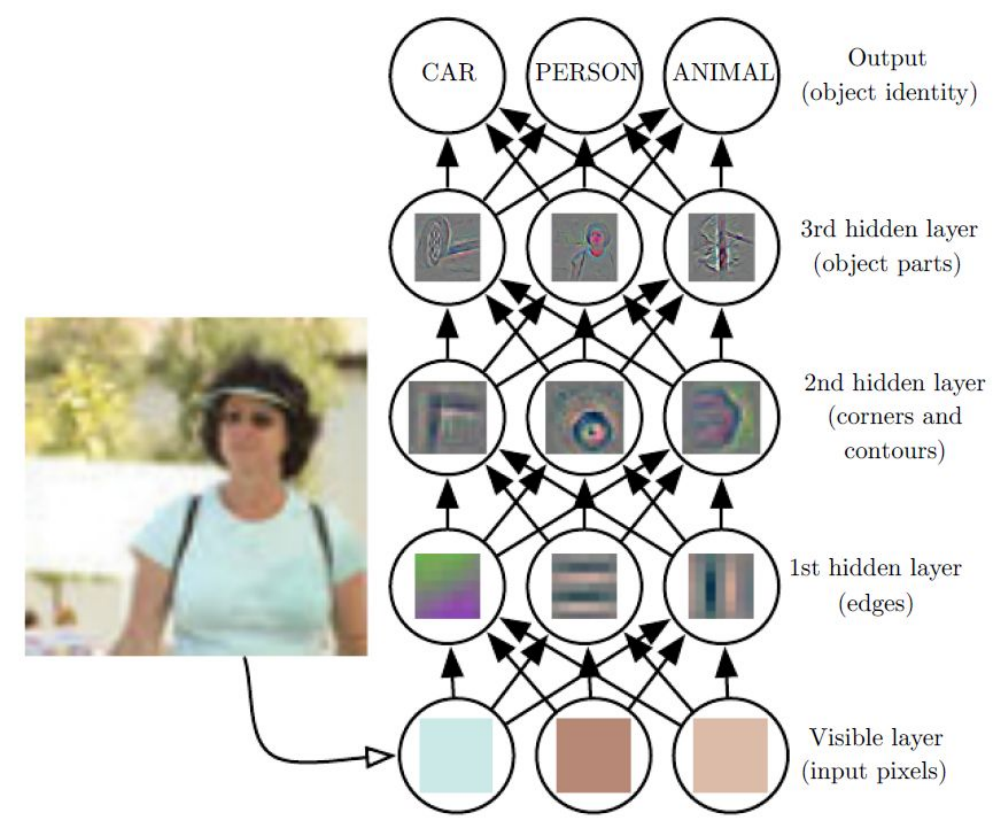
Activation Function: Mathematical gate(operation) applied after the sum → without it, the neuron is just doing basic math functions (like mult. adding), which means the whole network, regardless of how many layers, can only learn linear relationships.

- **Introduces non-linearity**, helping learn complex patterns

Output: single number that comes out and gets passed to the next layer



1. **Visible Layer**(input pixels) → raw image is broken down into pixels. The colored squares at the bottom are just patches of color. The networks sees numbers, not the photo (think as if we are breaking a painting into individual dots of paint)
2. **Hidden Layer** (edges) →first layer learns to spot basic edges and lines. Horizontal lines, vertical lines, diagonal lines (nothing really important here, just indicates a color changed occurred)
3. **2nd Hidden layer** (corners and contours) → second layer combines those edges into shapes, it can see curves, corners, and outlines. The network can start to see certain edge combinations that form a circle or a square
4. **3rd Hidden layer** (Object Parts) → combines those shapes into recognizable pieces → eyes, wheels, fur patches. It does not know what a "car" is yet, but the network is recognizing it is seeing a circular shape attached to a rectangular shape
5. **Output Layer:** Combines everything from the last hidden layer and makes a final decision → CAR, PERSON, or ANIMAL, assigns a probability value to each and picks the highest value



This a more advanced type of neural network (for images)

Input → full image is too large to feed directly into a basic neural network

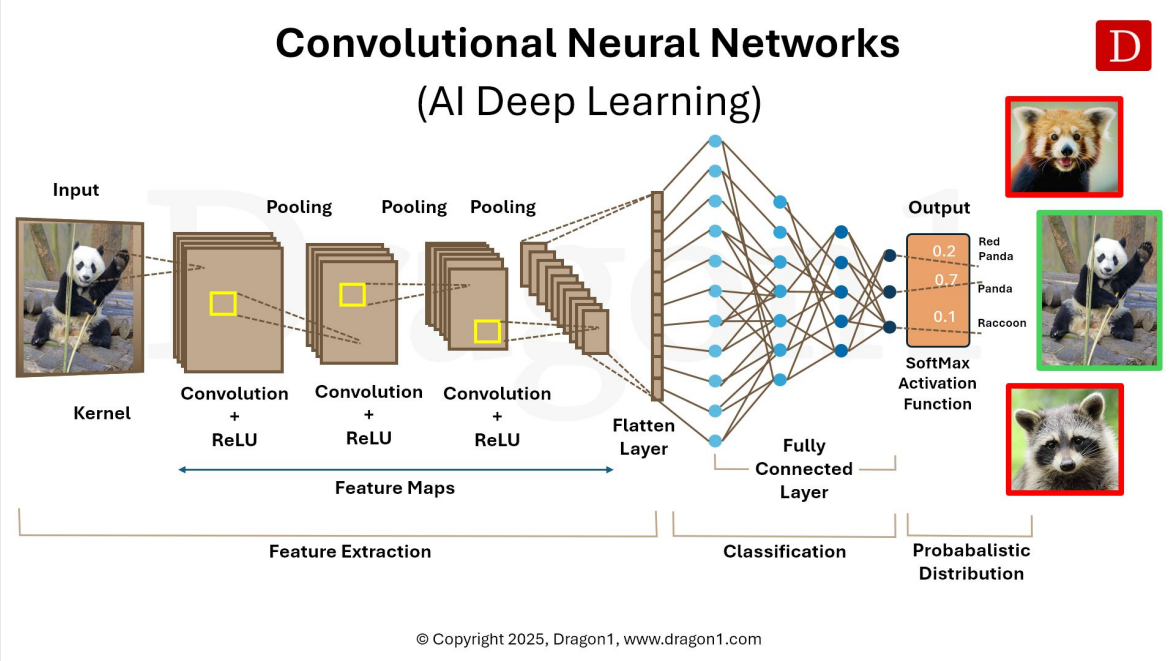
Kernel → a small sliding window(3x3 square) that moves across image scanning for patterns. Think of it like a flashlight scanning a dark room, one small patch at a time

Convolution + ReLU → kernel slides across the image, does a weighted sum of each patch, the ReLU filters out anythin negative → result is a **feature map** → a condensed version of the image that highlights where certain patterns appeared

Pooling: makes the feature map smaller, keeping the strongest signal in each region → network becomes smaller + faster, less sensitive to exactly where a pattern appears

Repeats 3x → each round of convolution + pooling gives a smaller more abstract feature map → go from raw pixels to high-level features

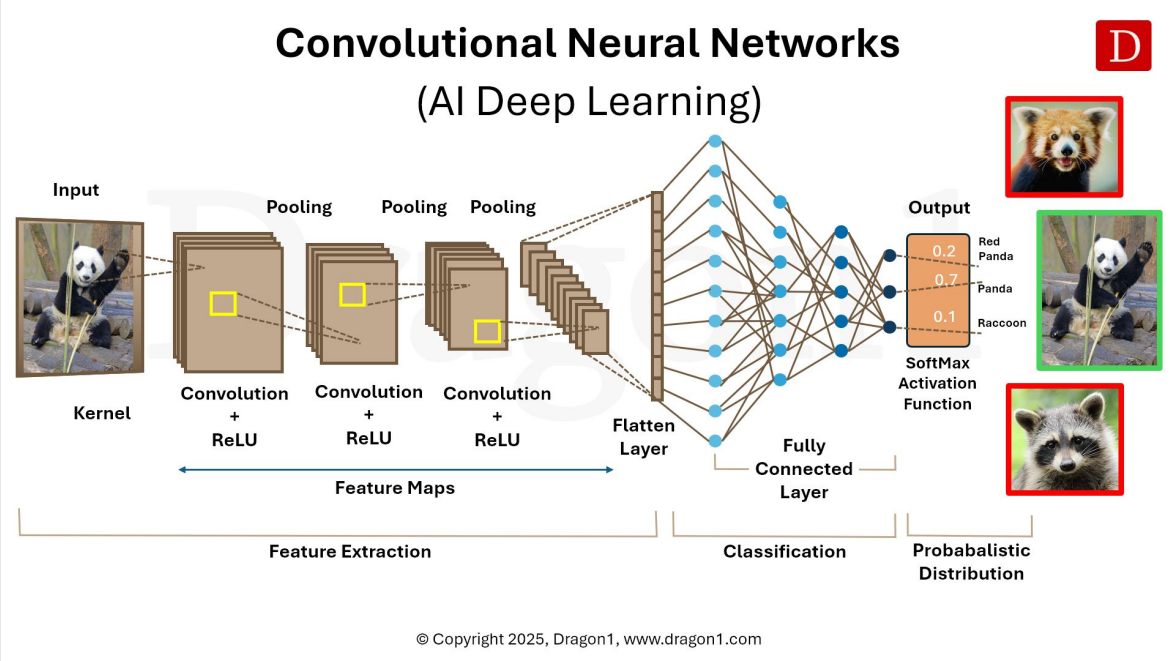
Flatten Layer → takes 3D feature maps + compresses them into a single long list of numbers (so a regular NN can process it)



Fully Connected Layer → what we building today, every neuron connects to every neuron in next layer → takes extracted features → combines them into class predictions

SoftMax Activation function → like sigmoid but for more than 2 classes (converts raw input numbers into probabilities)

Output → highest probability shows



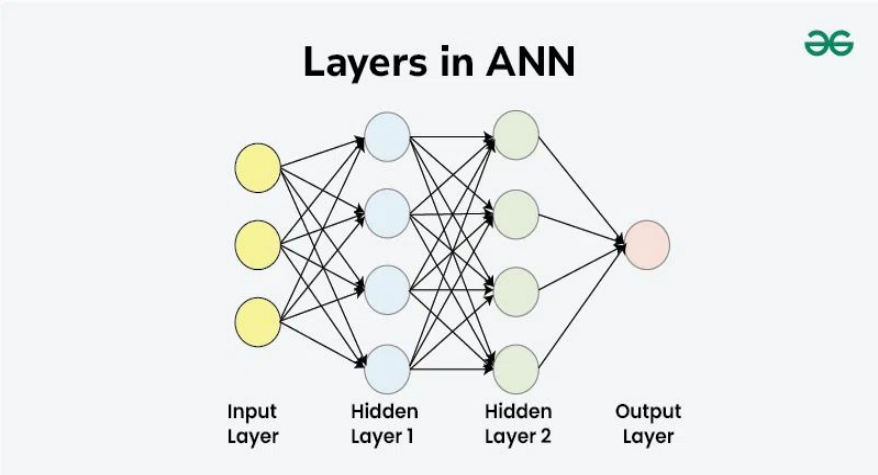
The sigmoid in a neural network is a math function that compresses any input to a value between 0 and 1, giving an S-shaped curve that's often used to represent probabilities.

Layers: How a network is structured?

Input layer: not really a layer of neurons, just raw data (ex: data set has 30 features(measurements), layer has 30 nodes, one per feature)

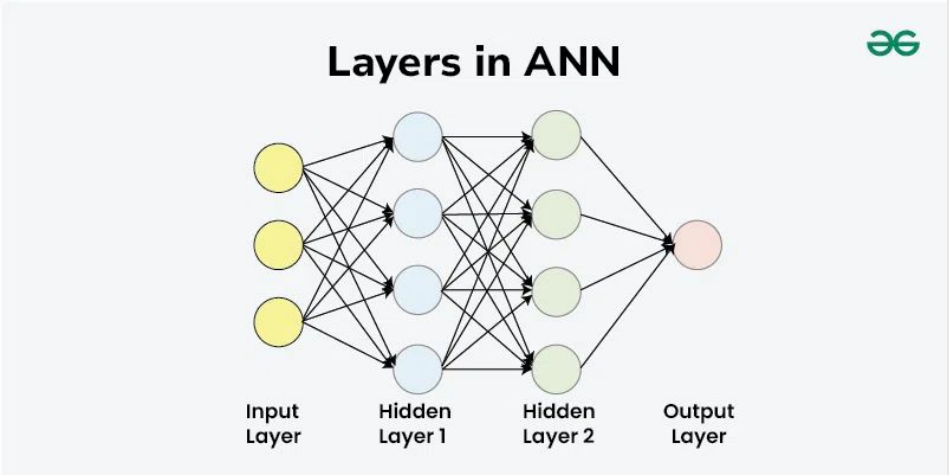
Hidden Layers: learning occurs here, each neuron in a hidden layer receives output from every neuron in the previous layer, weighted sum + activation → passes result forward.

- **First hidden layer might learn simple patterns**(if measurement value is abnormal or not?) **deeper layers** combine simple patterns into more complex ones



Layers: How a network is structured?

Output Layer: Final layer, for binary classification (yes/no), there is 1 neuron here. Uses sigmoid activation to output a number between 0 and 1, which is interpreted as a probability. Above 0.5 → predict class is 1, below 0.5 → predict class is 0



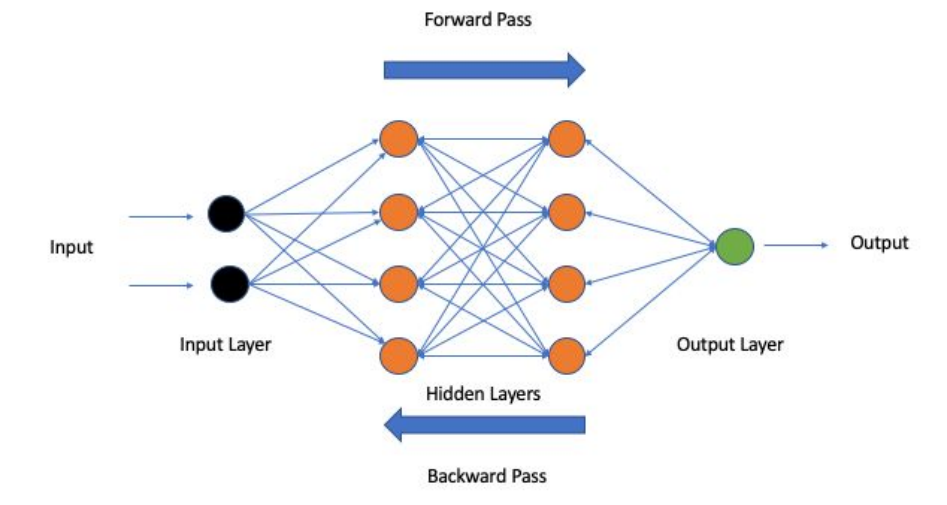
Layers: Key terms to know

Forward Pass: Process of data moving left to right through all layers to produce a prediction (do this every time you want the network to make a guess)

Loss: after the forward pass, compare prediction to true answer, loss is score for how wrong the network is

Backpropogation: after computing loss, run backwards through network to figure out which weights caused the e

Epoch: one complete pass through entire training dataset, typically run hundreds or thousands of these



What you need to build One? (tools, libraries, key programming concepts)

Python: readable + has tons of libraries

Numpy - numerical python library → NN is matrix math (multiplying big grids of math together), fast + efficient

Scikit-learn: python ML library → loading clean dataset, splitting data into train + test sets, and normalizing data with *StandardScaler*

Matplotlib →python library: use to plot loss over time (visualize if network is learning or not)

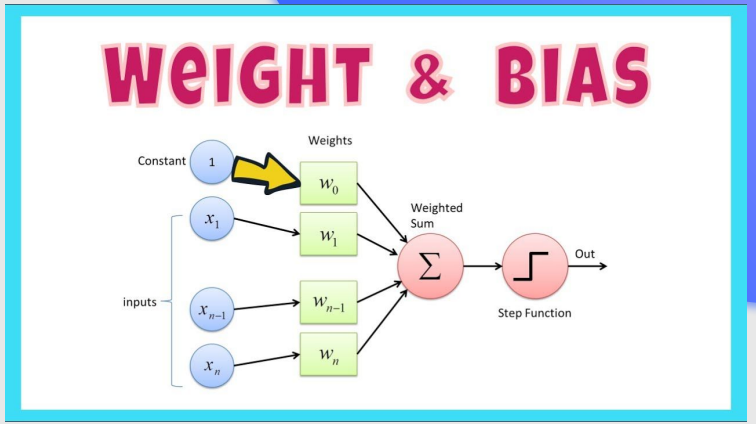
Google Colab - free jupyter notebook in browser w/ python + libraries pre-installed

Key Concepts

Weights & Biases: actual parameters network learns, before training they are random, after, they encode everything the network has learned (ex: 1 billion parameters → 1 billion weights and biases)

Activation Function: mathematical gate that introduces non-linearity → without it, would result in a single linear equation → important for complex calculations/patterns

Loss Function: measures how bad the network prediction was (lower value means confident + correct and vise versa) → training reduces this value

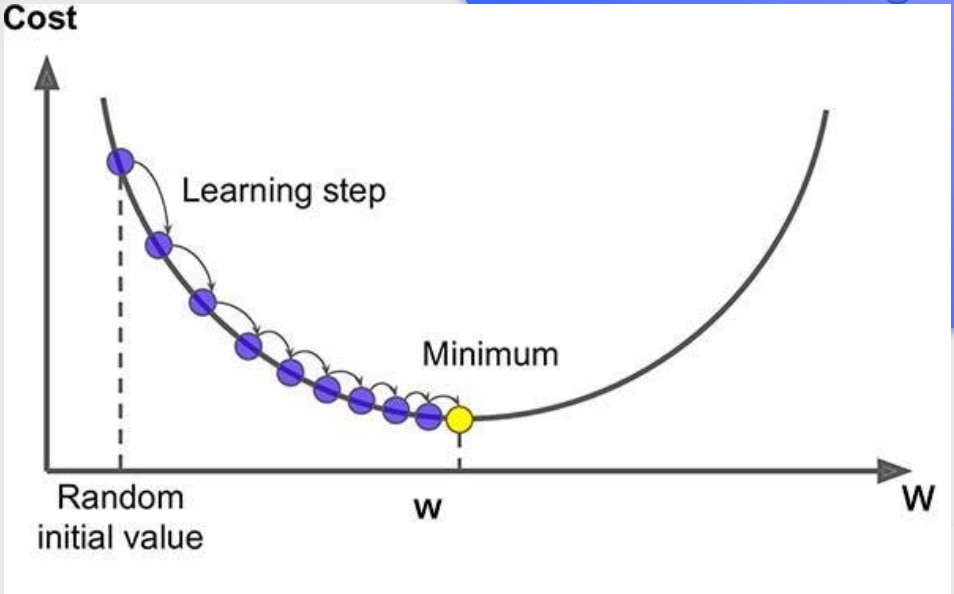


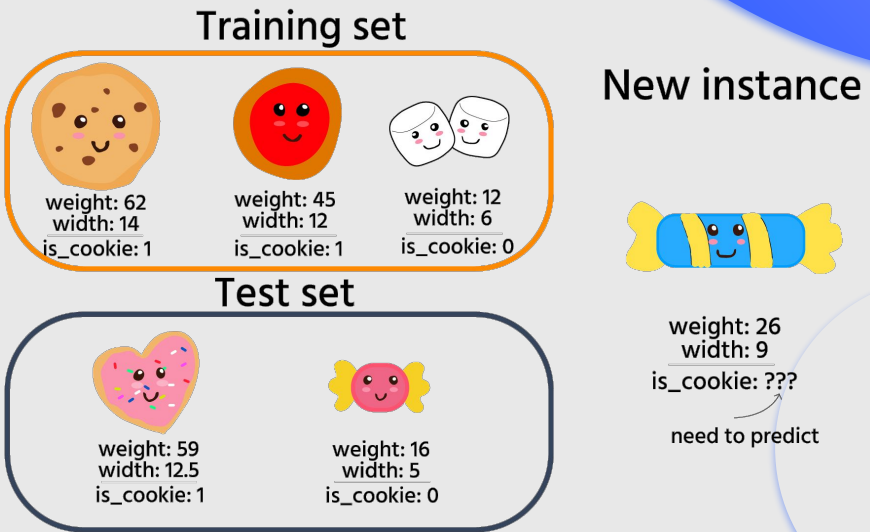
Key Concepts

Gradient Descent: algorithm to reduce loss, gradient refers to slope (which direction is uphill for loss) → refers to going opposite direction to find lowest point

Learning Rate: controls how big each step is in gradient descent (too large: overshooting min + loss bounces; too small: takes too long to converge)

- 0.1 is common starting point. This value is constantly altered, part of training the network



Key Concepts

Train/TestSplit - never eval your model on data it trained on, hold out 20% of your data before training starts → held out set is the true test of whether the model learned